A

Major Project Report

On

IMPLEMENTING SMART CONTRACTS FOR AUTOMATED AND TRANSPARENT SUPPLY CHAIN MANAGEMENT IN AGRICULTURE

Submitted to JNTU HYDERABAD

In Partial Fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted

By

BURRA MALATHI	**(218R1A1213)**
BOKKENA SANGHAVI	**(218R1A1212)**
BATHULA CHARAN THEJA	**(218R1A1211)**
GADDE SHIVA	**(218R1A1222)**

Under the Esteemed guidance of

Mrs. K. SUSHMA

Assistant Professor, Department of IT

Department of Information Technology

CMR ENGINEERING COLLEGE

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

(2024-2025)

CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist, Hyderabad-501 401)

Department of InformationTechnology

CERTIFICATE

This is to certify that the project entitled **"IMPLEMENTING SMART CONTRACTS FOR AUTOMATED AND TRANSPARENT SUPPLY CHAIN MANAGEMENT IN AGRICULTURE"** is a bonafide work carried out by

BURRA MALATHI	**(218R1A1213)**
BOKKENA SANGHAVI	**(218R1A1212)**
BATHULA CHARAN THEJA	**(218R1A1211)**
GADDE SHIVA	**(218R1A1222)**

in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide
Mrs. K. SUSHMA
Assistant Professor
Department of IT
CMREC
Hyderabad

Head of the Department
Dr. MADHAVI PINGILI
Professor & HOD
Department of IT
CMREC
Hyderabad

Project External Examiner

DECLARATION

This is to certify that the work reported in the present project entitled **"IMPLEMENTING SMART CONTRACTS FOR AUTOMATED AND TRANSPARENT SUPPLY CHAIN MANAGEMENT IN AGRICULTURE"** is a record of bonafide work done by us in the Department of Information Technology, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

BURRA MALATHI (218R1A1213)

BOKKENA SANGHAVI (218R1A1212)

BATHULA CHARAN THEJA (218R1A1211)

GADDE SHIVA (218R1A1222)

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Madhavi Pingili**, Professor & HOD, **Department of IT, CMR Engineering College** for their constant support.

We are extremely thankful to **Mrs. K. SUSHMA,** Assistant Professor, Internal Guide, Department of IT, for her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, We are very much thankful to our parents who guided us for every step.

BURRA MALATHI (218R1A1213)

BOKKENA SANGHAVI (218R1A1212)

BATHULA CHARAN THEJA (218R1A1211)

GADDE SHIVA (218R1A1222)

CONTENTS

ABSTRACT

The "Implementing Smart Contracts for Automated and Transparent Supply Chain Management in Agriculture" project addresses the critical need for transparency and accountability in the agricultural supply chain. As the global demand for sustainable and ethically sourced products rises, ensuring the authenticity and origin of agricultural products becomes paramount. Tracing of goods in the agricultural field requires gathering, communicating and maintaining critical data by specifically identifying the source, multiple data exchanges in the logistic network. This project leverages block chain technology to establish an immutable and decentralized ledger that traces the entire lifecycle of agricultural products, from cultivation to consumption. The primary objectives of the project include the implementation of a block chain - based trace ability system that enables stake holders, including farmers, distributors, retailers, and consumers, to access real-time, verifiable information about the production and distribution processes. The use of smart contracts facilitates automated verification and validation of key milestones, ensuring data integrity and reducing the risk of fraud or misrepresentation in the supply chain. Through the integration of block-chain technology, the project aims to empower consumers with a trustworthy and transparent view of the journey of agricultural products, fostering informed choices that align with sustainability goals. As conventional agri-food logistic practices can no longer satisfy consumer demands, developing a traceability frame work for agri-food supply is becoming increasingly urgent. Additionally, the system benefits farmers by providing a secure and tamper-proof record of their contributions to sustainable practices, enhancing market competitiveness and supporting fair trade. As the project evolves, it has the potential to revolutionize the way stake holders engage with agricultural products, fostering a greater sense of trust, environmental responsibility, and social awareness in the pursuit of sustainable agriculture.

Keywords: Block chain, Ethereum, Smart contract, Decentralized ledger, Integration, Sustainability, Transparency.

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

Agriculture is a cornerstone of global food security and economic stability, providing livelihoods to millions of people worldwide. However, the agricultural supply chain is often plagued by inefficiencies, lack of transparency, and fraudulent activities. Traditional supply chain models rely heavily on manual processes, paper-based record-keeping, and multiple intermediaries, leading to delays, increased operational costs, and trust issues among stakeholders. Farmers, suppliers, distributors, and retailers frequently face challenges related to delayed payments, mismanagement of resources, and counterfeiting of agricultural products. These inefficiencies not only affect profitability but also compromise the quality and safety of agricultural goods, ultimately impacting consumers. To address these challenges, the adoption of block chain-based smart contracts has emerged as a revolutionary solution for enhancing transparency and automation in agricultural supply chains. Smart contracts are self-executing agreements with terms directly written into code, ensuring that transactions are automatically executed when predefined conditions are met. By integrating smart contracts into supply chain management, various processes such as payments, product tracking, and compliance verification can be streamlined, reducing reliance on intermediaries and mitigating the risk of human error. This leads to greater efficiency, reduced costs, and improved trust among all parties involved. One of the key benefits of implementing smart contracts in agricultural supply chains is real-time traceability. Blockchain technology enables all stakeholders to access an immutable, decentralized ledger that records every transaction, from production and transportation to final delivery. This enhances accountability and allows consumers to verify the origin and quality of agricultural products. Additionally, integrating smart contracts with Internet of Things (IoT) devices can further improve supply chain visibility by providing real- time data on storage conditions, transportation status, and product authenticity. By ensuring that every step of the supply chain is recorded and verified, smart contracts help prevent fraud, reduce waste, and optimize resource allocation. This project aims to explore the practical implementation of smart contracts in agricultural supply chain management, highlighting their potential to automate operations, enhance transparency, and build trust among stakeholders. By leveraging block chain technology, IoT integration, and automated contract execution, this research will demonstrate how digital innovation can revolutionize traditional agricultural supply chains, ensuring efficiency, security, and long-term sustainability.

1.2 Project Objectives

The objective of this project is to implement smart contracts for automated and transparent supply chain management in agriculture using blockchain technology. This aims to enhance efficiency, security, and trust among stakeholders by automating key processes such as order management, payments, and product verification. It focuses on automating payments, reducing disputes, ensuring fair trade for farmers, and enhancing regulatory compliance.

1.3 Purpose of the project

The purpose of this project is to modernize agricultural supply chain management by leveraging smart contracts and blockchain technology to enhance automation, transparency, and security. Traditional supply chains suffer from inefficiencies, fraud, and lack of traceability, which this project aims to address by eliminating intermediaries, ensuring real-time tracking of goods, and automating transactions and payments. By integrating IoT devices for monitoring and smart contracts for self-executing agreements, the project seeks to reduce operational costs, prevent disputes, and empower farmers with fair trade opportunities. Ultimately, this initiative aims to create a trustworthy, efficient, and sustainable agricultural ecosystem for all stakeholders.

1.4 Existing System with Disadvantages

Before exploring how smart contracts can transform the agricultural supply chain, it's essential to understand the current systems and technologies in place. Traditionally, the agricultural supply chain is complex, involving multiple stakeholders such as farmers, suppliers, distributors, transporters, retailers, and consumers. However, the existing systems in the agriculture supply chain are often fragmented, inefficient, and prone to issues such as fraud, delays, lack of transparency, and high transaction costs.In many rural or less developed areas, the agricultural supply chain still relies on manual processes. These systems often involve paper-based contracts, manual invoicing, and physically signed agreements. The flow of goods and payments is tracked using paper documents such as bills of lading, delivery receipts, and invoices.

Disadvantages

- **Lack of Transparency and Trust:** Traditional systems are often opaque, with limited visibility into each stage of the supply chain. This leads to a lack of trust between producers, suppliers, and consumers.

- **Manual Processes and Errors:** Much of the agricultural supply chain still relies on manual processes, increasing the risk of errors, fraud, and delays.
- **Increased Costs:** Due to inefficiencies, intermediaries, and a lack of automation, costs associated with logistics, paperwork, and payments can be high.
- **Difficulty in Verifying Product Quality:** Ensuring the quality and authenticity of agricultural products is often challenging, especially for consumers who want to verify the origin or sustainability of the products they purchase.
- **Slow Payment Settlements**: Payment processing in the agricultural supply chain can be slow and inefficient, particularly when multiple intermediaries are involved.

1.5 Proposed System with Features

The proposed system aims to address the inefficiencies, lack of transparency, and trust issues in the agricultural supply chain by implementing a blockchain-based platform with smart contracts. This system will automate key processes, ensure transparency, and facilitate real-time tracking of agricultural goods from the farm to the consumer. By using distributed ledger technology (DLT) and smart contracts, the system will significantly improve coordination among stakeholders and streamline operations. A decentralized, distributed ledger that records every transaction, product movement, and contract between parties in the supply chain. This ensures that data is transparent, immutable, and verifiable by all participants.

Advantages

- **Increased Transparency and Trust**
    - **Immutable Records**: Every transaction, movement of goods, and contract term is recorded on the blockchain, which is tamper-proof and visible to all stakeholders. This eliminates the need for intermediaries and increases trust between participants.
- **Enhanced Security**
    - **Blockchain Security**: The decentralized nature of blockchain ensures that data is stored across multiple nodes, making it resistant to tampering or hacking. All participants have access to the same data, ensuring that no single party can manipulate the records.
- **Improved Product Quality Assurance**
    - **Automated Quality Checks**: With IoT and blockchain, each batch of products is continuously monitored for quality, and the results are automatically logged.

- **Certification Verification**: The blockchain can record certifications (e.g., organic, fair trade), making it easier for consumers to verify the authenticity and ethical standards of agricultural products.
- **Consumer Confidence**
    - **Enhanced Consumer Trust**: Consumers increasingly demand information on where their food comes from, how it was produced, and whether it is safe and sustainable. With blockchain, consumers can verify these aspects at the point of sale.

1.6 Input and Output Design

Input Design

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy.

Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

Objectives

- Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.
- It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

Output Design

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

- Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
- Select methods for presenting information.
- Create document, report, or other formats that contain information produced by the system.
- The output form of an information system should accomplish one or more of the following objectives.
- Convey information about past activities, current status or projections of the Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

2. LITERATURE SURVEY

X. Wang and X. Bi, "Application of BIM Algorithm and Block Chain Technology in the Construction of Agricultural Safety Traceability System,"2024 International Conference on Distributed Computing and Optimization Techniques (ICDCOT), Bengaluru, India, 2024, pp.1-6,doi: 10.1109/ICDCOT61034.2024.10515791. [1] The quality and safety issues of agricultural products are related to national food security and affect the quality of life of the general public. The BIM model algorithm and block chain consensus algorithm construct an agricultural security traceability platform. The BIM model has a clear architecture and obvious visualization effect in system construction, block chain technology contributes to the improvement of system processes and traceability methods.

M. Musthafa Sheriff and D. John Aravindhar, "Integrated Block chain-Based Agri- Food Traceability and Deep Learning for Profit-Optimized Supply Chain Management in Agri-Food Supply Chains,"2024 Fourth International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2024, pp.1-6, doi: 10.1109/ ICAECT 60202.2024.10469024. [2] The integration of block chain technology and Deep Belief Network- based secure Block Chain Management (DBCM) with the advanced hybrid crypto graphic mechanism HEDS presents a promising solution to the complex challenges encountered in agri- food supply chains (AFSC). This innovative approach ensures agri-food safety for consumers and increased profitability for farmers.

A. Toke, M. F. Monirand T. Ahmed, "Blockchain in Agriculture: A Scalable Solution for Crop Quality Assessment and Data Integrity," ***2024 IEEE International Black Sea Conference on Communications and Networking (Black Sea Com),*** **Tbilisi, Georgia, 2024, pp. 205-210, doi: 10.1109/BlackSeaCom61746.2024.10646243. [3]** The authors proposed a scalable block chain-based framework that enhances transparency, traceability, and security in agricultural supply chains. The study highlights how smart contracts and decentralized ledgers can prevent data tampering, ensuring accurate and verifiable records of crop quality. Additionally, it discusses challenges such as scalability, adoption barriers, and integration with existing agricultural technologies, providing insights into the future of block chain in agribusiness.

L.Ma, F.Gaoand X.Li,"Harnessing Block chain for Agricultural Product Traceability," ***2024 4th International Conference on Computer Science and Block chain (CCSB),*** **Shenzhen, China, 2024, pp.507-514, doi:10.1109/ CCSB63463.2024.10735594. [4]** To enhance the safety of

agricultural product supply chains and address the issue of poor traceability, this article integrates block chain technology with agricultural product supply chains to design a block chain-based traceability system for agricultural products. On this basis, the article proposes an “on-chain” and “off-chain” data storage method to ensure efficient data queries. To improve the credibility of the traceability system, the article also suggests using smart contracts and encryption technology to protect private data and traceability information, ensuring the security of various data.

B.H.R.D.Reddy, A.K.Veeduand B.BM, "Block chain Technology in Agriculture," ***20248th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS),*** **Bengaluru, India, 2024,pp. 1- 7, doi: 10.1109/ CSITSS64042.2024.10816748. [5]** This study addresses challenges in the agricultural sector by leveraging block chain technology to assist farmers in generating revenue and enhancing the economy. Using Solidity, the team has developed smart contracts to facilitate secure transactions among farmers, distributors, retailers, and consumers, ensuring fair remuneration for farmers and fostering balanced profit distribution across the supply chain. Front-end development utilizing React has provided an engaging user interface.

Block chain technology for sustainable supply chains: A comprehensive review and future prospects March 2024 World Journal of Advanced Research and Reviews 21(3): 980-994 DOI: 10.30574/ wjarr.2024.21.3.0804. [6] By examining successful implementations, challenges faced, and future trends, it has contributed valuable insights to the discourse on leveraging block chain for sustainability. It highlights the practical applications of block chain, including transparency, traceability, ethical sourcing, and smart contracts for sustainable practices.

K. R. Chaganti, B. N. Kumar, P. K. Gutta, S. L. Reddy Elicherla, C. Nagesh and K. Raghavendar, "Blockchain Anchored Federated Learning and Tokenized Traceability for Sustainable Food Supply Chains," ***2024 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS),*** **Gobichettipalayam, India, 2024,**

pp. 1532-1538, doi: 10.1109/ICUIS64676.2024.10866271. [7] The authors propose a decentralized framework that ensures data privacy while enabling real-time tracking and authentication of food products. By leveraging federated learning, the system allows multiple stakeholders to collaboratively train machine learning models without sharing sensitive data, improving predictive analytics and decision-making.

M. Alsamman, Y. Fazea and F. Mohammed, "Towards a Blockchain-Based Agricultural Ecosystem: A systematic review," ***2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*****, Sakheer, Bahrain, 2023,pp. 381-388, doi: 10.1109/3ICT60104.2023.10391332. [8]** It explores the potential of block chain technology in transforming the agricultural sector. The study systematically reviews existing research to highlight block chain's role in enhancing transparency, security, and efficiency in agricultural supply chains. The authors discuss various applications, challenges, and future directions for integrating block chain into agriculture, emphasizing its impact on trust, traceability, and smart contracts.

Subramanian, B. Selvaraj, R. Sivakumar, R. Tabassum and S. Rajaram, "Enhancing Supply Chain Traceability with Block chain Technology: A Study on Dairy, Agriculture, And Sea food Supply Chains,"2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 2023, pp.967-971, doi:10.1109/ ICPCSN58827.2023.00165. [9] The potential use of block chain technology improves traceability and accountability in the food supply chain. It examines three key areas of the supply chain - dairy, agriculture, and sea food – and identifies the challenges associated with each, such as food safety, sustainability, and labor practices.

P. V. Chate, S. B. Mane and R. Y. Sarode, "An Efficient Approach For Sustainable Agricultural Trading Using Block chain Technology," 2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS), Kochi, India, 2022. [10] The authors have analyzed different works of literature with ground-level issues and proposed implementation aspects through high-level architecture design. The proposed model discusses the solution to eliminate middlemen costs that are indirectly paid by farmers and to connect buyer and seller directly without including any third party. It briefs how block chain can prevent possible security threats in E-agriculture.

Singh Bist et al., "AI-Enabled Block chain for Supply Chain in Agriculture," 2022 IEEE Creative Communication and Innovative Technology (ICCIT), Tangerang, Indonesia, 2022, pp.1-5, doi: 10.1109/ICCIT55355.2022.10118743. [11] Block chain provides a decentralized data structure to store and retrieve shared data with multiple entrusted parties. The technology offers several advantages, including laminating the dependency on the centralized supply chain

management database, which is also referred to as single point failure, and the high cost due to verifying and monitoring transactions by a third-party user.

Shivendra, K. Chiranjeevi, M. K. Tripathi and D. D. Maktedar, "Block chain Technology in Agriculture Product Supply Chain," 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), Coimbatore, India, 2021, pp. 1325- 1329, doi:10.1109/ICAIS50930.2021.9395886. [12] The generic framework and solutions are proposed using smart technologies and block chain to control, track and execute company operations eliminating intermediaries and the main processing point for crop price traceability in the agricultural supply chain.

G. K. Akella, S. Wibowo, S. Grandhi and S. Mubarak, "Design of a Block chain-based Decentralized Architecture for Sustainable Agriculture: Research-in-Progress," ***2021 IEEE/ACIS 19th International Conference on Software Engineering Research, Management and Applications (SERA),*** **Kanazawa, Japan, 2021, pp. 102-107, doi: 10.1109/SERA51205.2021.9509044. [13]** This study explores the development of a block chain-based decentralized framework to promote sustainability in agriculture. The authors proposed an architecture that enhances transparency, security, and efficiency in agricultural processes by leveraging block chain's immutable ledger and decentralized nature. The study discusses how smart contracts can automate agricultural transactions, ensuring fair trade and reducing intermediaries. The study provides initial insights into block chain's role in sustainable agriculture and outlines future research directions for refining the proposed framework.

Kang, P., & Indra-Payoong, N. (2021). A Framework of Block chain Smart Contract for Sustainable Agri-Food Supply Chain. ***INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND TECHNOLOGY (ISJET), 5*****(2), 1–14. [14]** This research determined to seek out a framework based on block chain, primarily due to its data immutability and transparency that provide outstanding rewards to willing participants of this experimental system coupled with smart contract in order to save costs and increase efficiency. With the framework, it hopes to re- empower the rights and competitiveness of food producers, as well as leveling the playing field, so to speak, in the agri-food supply chain. This would not just benefit all the stakeholders of the industry; it would also be exceedingly favorable to the end consumers and sustainability as a whole.

Blockchain in agriculture January 2021 DOI:10.1016/B978-0-12-821470-1.00003-3. [15] It aims to establish a proven and trusted environment to build a transparent and more sustainable food production and distribution, integrating key stakeholders into the supply chain. It proposed the general goals of optimizing the competitiveness and ensuring the sustainability of the agri-food supply chain, as well as designing a clear regulatory frame work for block chain implementations.

C. YU, Y. ZHAN and Z. LI, "Using Block chain and Smart Contract for Traceability in Agricultural Products Supply Chain," ***2020 International Conference on Internet of Things and Intelligent Applications (ITIA),*** **Zhenjiang, China, 2020, pp. 1-5, doi: 10.1109/ITIA50152.2020.9312315. [16]** This explores how block chain and smart contracts can enhance traceability in agricultural supply chains. The authors proposed a decentralized framework that ensures transparency, security, and trust among stakeholders by recording transactions on an immutable ledger. The study highlights the advantages of using smart contracts to automate processes such as quality verification, logistics tracking, and compliance checks. It discusses implementation challenges, including scalability and integration with existing agricultural systems, while emphasizing the potential of block chain to improve food safety and consumer confidence.

W. Lin *et al.,* "Block chain Technology in Current Agricultural Systems: From Techniques to Applications," in *IEEE Access,* vol. 8, pp. 143920-143937, 2020, doi: 10.1109/ ACCESS.2020.3014522. [17] The authors analyzed various block chain techniques, including consensus mechanisms, smart contracts, and decentralized storage, and explore their practical applications in agriculture. This highlights key benefits such as enhanced traceability, data security, and supply chain transparency while also addressing challenges like scalability, high implementation costs, and integration with existing agricultural technologies.

Demestichas, Konstantinos P., Nikolaos Peppes, Theodoros Alexakis and Evgenia F. Adamopoulou. "Blockchainin Agriculture Traceability Systems: A Review." Applied Sciences (2020): n. pag. DOI:10.3390/app10124113 Corpus ID: 225742837. [18] The usage of block chains can advantageously help to achieve traceability by irreversibly and immutably storing data. Block chain Technology creates a unique level of credibility that contributes to a more sustainable food industry. Block chain could also facilitate insurance programs for securing farmers (i.e. members of the cooperatives) against unpredicted weather conditions that affect their crops or other risks such as natural disasters.

K. Salah, N. Nizamuddin, R. Jayaraman and M. Omar, "Blockchain-Based Soybean Traceability in Agricultural Supply Chain," in *IEEE Access*, vol. 7, pp. 73295-73305,2019, doi: 10.1109/ ACCESS.2019.2918000. [19] The authors propose a block chain-based framework that ensures transparency, security, and efficiency in tracking soybean production, processing, and distribution. By leveraging block chain's decentralized and immutable ledger, the system prevents data tampering, enhances trust among stakeholders, and improves compliance with regulatory standards. The study also discusses smart contracts to automate verification processes, reducing manual intervention and operational inefficiencies.

J. Li and X. Wang, "Research on the Application of Block chain in the Traceability System of Agricultural Products," *2018 2nd IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*, Xi'an, China, 2018, pp. 2637-2640, doi: 10.1109/IMCEC.2018.8469456. [20] This study discusses how block chain can address challenges such as data tampering, fraud, and inefficiencies in traditional traceability systems. By leveraging decentralized and immutable ledger technology, the proposed system ensures real-time monitoring and authentication of agricultural products from production to consumption. The study also examines the integration of smart contracts to automate verification processes, reducing reliance on intermediaries.

3. SOFTWARE REQUIREMENTS

3.1 Problem Statement

The agricultural supply chain faces inefficiencies, lack of transparency, fraud, and delayed payments, leading to financial losses and distrust among stakeholders. Manual processes and intermediaries create bottlenecks, while the absence of real-time traceability results in counterfeit products and disputes over quality and delivery. To address these issues, this project proposes implementing smart contracts on a blockchain-based platform to automate transactions, ensure secure record-keeping, and improve trust. This solution aims to eliminate fraud, enhance efficiency, and create a transparent, fair, and sustainable agricultural supply chain.

3.2 Module and their Functionalities

Tensor Flow

Tensor Flow is a free and open-source software library for data flow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google. Tensor Flow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multi dimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object.
- Sophisticated (broadcasting) functions.
    - Tools for integrating C/C++ and Fortran code.
    - Useful linear algebra, Fourier transform, and random number capabilities.

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze.

Mat plot lib

Mat plot lib is a Python2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Mat plot lib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface tool kits. Matplot lib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

Sci kit–learn

Sci kit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python is an interpreted high-level programming language for general-purpose programming. Created by Guidovan Rossumand first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Install Python Step-by-Step in Windows and Mac

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3. Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices. Before you start with the installation process of Python. First, you need to know about your System Requirements.

My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows7 device or to install Python3. Download the Python Cheat sheet here. The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

Fig no: 3.2.1

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Fig no: 3.2.2

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

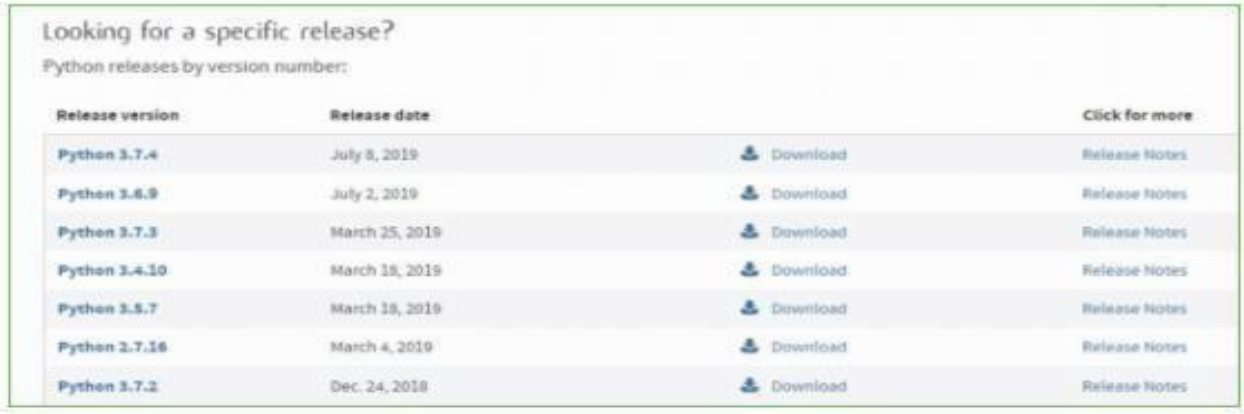


Fig no: 3.2.3

Step4: Scroll down the page until you find the Files option.

Step5: Here you see a different version of python along with the operating system.

Fig no: 3.2.4

- To download Windows 32-bit python, you can select anyone from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86web-based installer.
- To download Windows 64-bit python, you can select anyone from the three options:Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86- 64 web-based installer.

- Here we will install Windows x86-64 web-based installer. Here is your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation
- Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

- Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

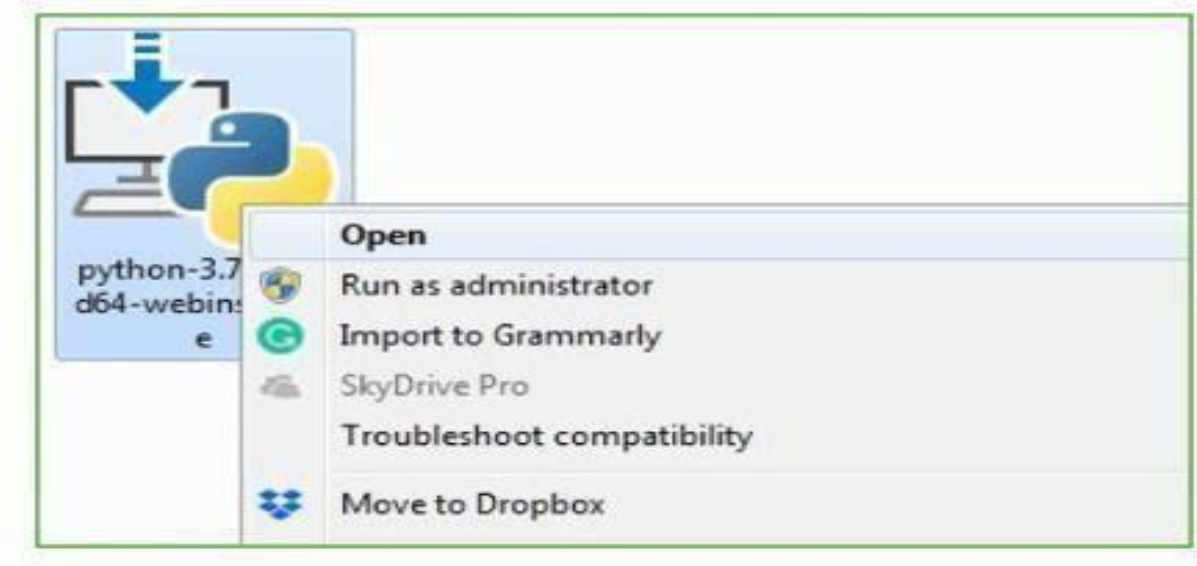


Fig no: 3.2.5

- Step 2: Before you click on Install Now, Make sure to put the tick on Add Python 3.7 to PATH.

Fig no: 3.2.6

- Step 3: Click on Install NOW After the installation is successful and click on the Close button.

Fig no: 3.2.7

- With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.
- Note: The installation process might take a couple of minutes.

Verify the Python Installation

- Step 1: Click on Start
- Step 2: In the Windows Run Command, type “cmd”.

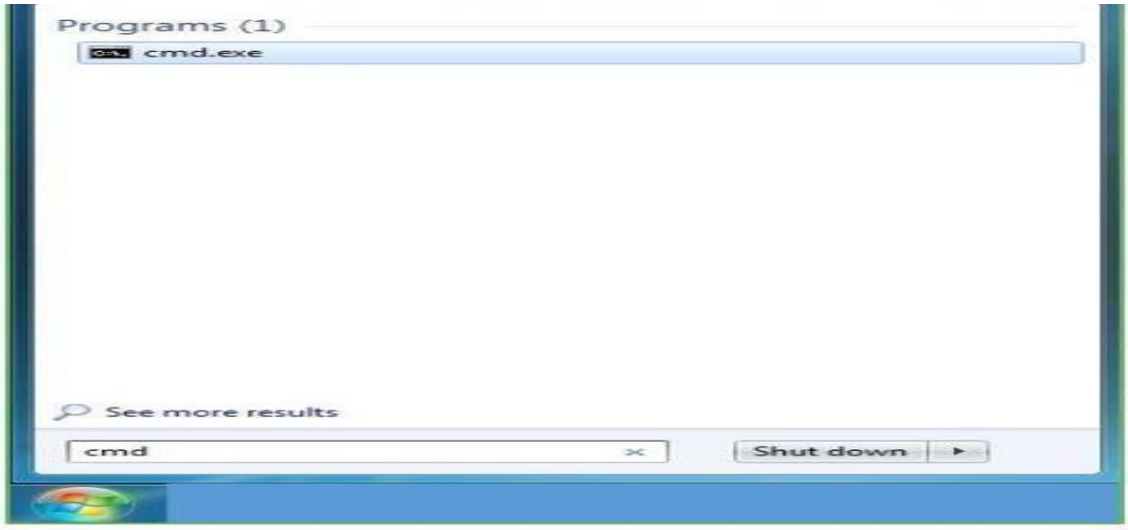


Fig no: 3.2.8

- Step 3: Open the Command prompt option.
- Step 4: Let us test whether the python is correctly installed. Type python–V and press Enter key.

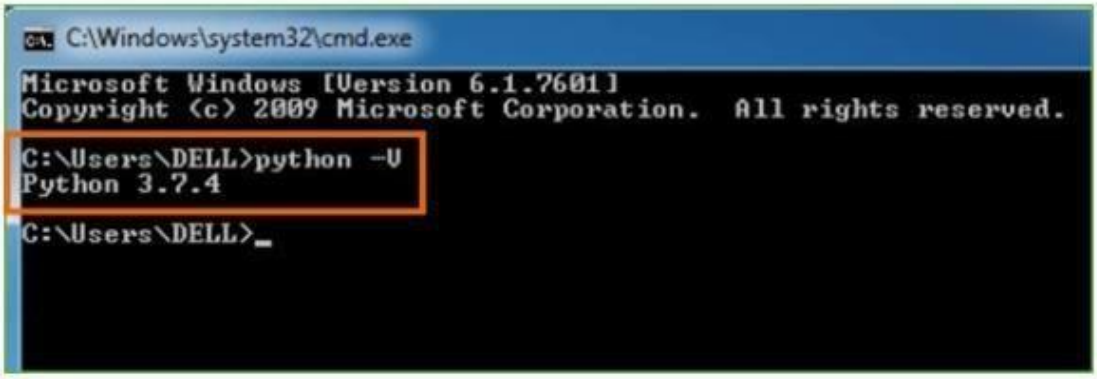


Fig no: 3.2.9

- Step 5: You will get the answer as 3.7.4
- Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

- Step 1: Click on Start
- Step 2:In the Windows Run command, type “python idle”.

Fig no: 3.2.10

- Step 3: Click on IDLE(Python 3.764-bit) and launch the program
- Step 4:To go ahead with working in IDLE you must first save the file. Click on File> Click on Save.

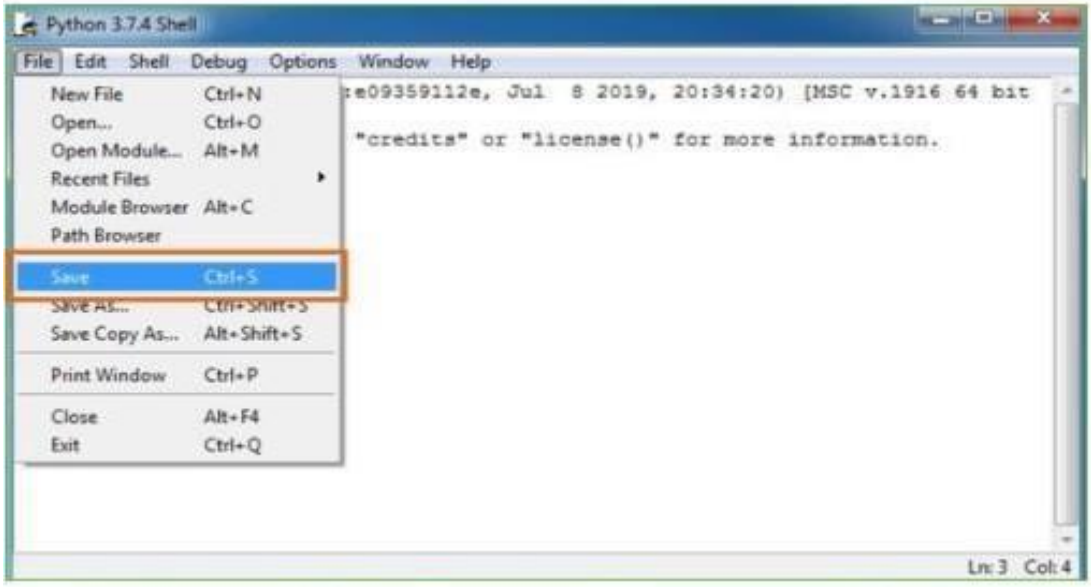


Fig no: 3.2.11

- Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.
- Step 6: Nowfore.g.enter print ("Hey World") and Press Enter.

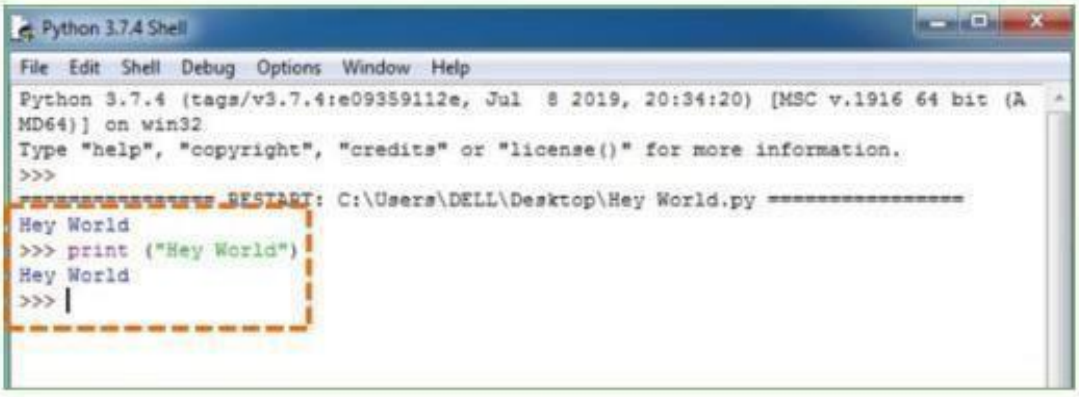


Fig no: 3.2.12

- You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.
- Note: Unlike Java, Python does not need semicolons at the end of the statements otherwise it won't work.

Django Framework:

What is Django?

Django is a Python framework that makes it easier to create websites using Python.

Django takes care of the difficult stuff so that you can concentrate on building your web applications.

Django emphasizes reusability of components, also referred to as DRY (Don't Repeat Yourself), and comes with ready-to-use features like login system, database connection and CRUD operations (Create Read Update Delete).

Django is a high-level Python web framework that follows the "don't repeat yourself" (DRY) principle and encourages rapid development of web applications. It provides a set of tools and functionalities that streamline common development tasks, allowing developers to focus more on building features rather than dealing with low-level details. Here are some key aspects and features of Django framework:

Model-View-Template (MVT) Architecture:

Django follows the MVT architecture pattern, which is similar to the Model-View-Controller (MVC) pattern. It separates the application's data (models), business logic (views), and presentation layer (templates), promoting code organization and maintainability.

ORM (Object-Relational Mapping):

Django includes a powerful ORM that abstracts database operations into Python objects, allowing developers to interact with databases using Python syntax rather than SQL queries. This simplifies database operations and reduces the amount of boilerplate code.

Admin Interface:

Django provides an admin interface out-of-the-box, which automatically generates a user-friendly interface for managing site content and data models. Developers can customize the admin interface to suit the application's needs.

URL Routing and View Handling:

Django uses a URL routing mechanism to map URLs to views (functions or classes) that handle HTTP requests. This makes it easy to define URL patterns and corresponding view functions for different parts of the application.

Template System:

Django's template system allows developers to create HTML templates with dynamic content using Django's template language. Templates support template inheritance, context variables, filters, and tags, making it easy to create reusable and modular templates.

Security Features:

Django includes built-in security features such as protection against common web vulnerabilities like CSRF (Cross-Site Request Forgery), XSS (Cross-Site Scripting), SQL injection, and click jacking.

Form Handling:

Django simplifies form handling by providing form classes that can be easily integrated into views. Form validation, data cleaning, and rendering of HTML forms are handled efficiently by Django's form handling features.

Middleware:

Django middleware allows developers to modify request and response objects, perform authentication, handle sessions, and implement custom middleware logic. Middleware provides a way to hook into the request-response cycle at various stages.

Built-in Development Server:

Django comes with a built-in development server that makes it easy to test and run Django applications locally during development.

Overall, Django's comprehensive feature set, robustness, scalability, and active community support make it a popular choice for developing web applications, including cognitive agent systems for tasks such as code retrieval from repositories.

Templates support template inheritance, context variables, filters, and tags, making it easy to create reusable and modular templates.

Django uses a URL routing mechanism to map URLs to views (functions or classes) that handle HTTP requests. This makes it easy to define URL patterns and corresponding view functions for different parts of the application.

How Django works:

In a cognitive agent system leveraging Django for code retrieval from repositories, Django's core

functionalities play a pivotal role. The system's backend, implemented in Django, handles user interactions, query processing, and integration with the code repository. Upon receiving search queries from the user interface, Django's controllers and views orchestrate the query processing flow, possibly incorporating natural language processing (NLP) techniques and semantic search algorithms. Django's ORM facilitates seamless interaction with the code repository database, enabling efficient retrieval and ranking of relevant code components. User feedback mechanisms, managed by Django, contribute to continuous learning and improvement of the system's relevance ranking algorithms. Finally, Django's presentation layer renders the processed search results, ensuring a smooth and intuitive user experience throughout the code retrieval journey.

Python Django is a web framework that allows to quickly creating efficient web pages. Django is also called batteries included framework because it provides built-in features such as Django Admin Interface, default database – SQLite3, etc. When you're building a website, you always need a similar set of components: a way to handle user authentication (signing up, signing in, signing out), a management panel for your website, forms, a way to upload files, etc. Django gives you ready- made components to use.

Most of the time when you'll be working on some Django projects, you'll find that each project may need a different version of Django. This problem may arise when you install Django in a global or default environment. To overcome this problem we will use virtual environments in Python. This enables us to create multiple different Django environments on a single computer. To create a virtual environment type the below command in the terminal**.**

In Django, each view needs to be mapped to a corresponding URL pattern. This is done via a Python module called URL Conf(URL configuration). Every URL Conf module must contain a variable url patterns which is a set of URL patterns to be matched against the requested URL. These patterns will be checked in sequence until the first match is found. Then the view corresponding to the first match is invoked.

3.3 Functional Requirements

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provides a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization
- Internal Outputs whose destination is within organization and they are the User's main interface with the computer
- Operational outputs whose use is purely within the computer department.

Input design is a part of overall system design. The main objective during the input design is as given below:

• To produce a cost-effective method of input.

• To achieve the highest possible level of accuracy.

• To ensure that the input is acceptable and understood by the user.

These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract.

These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

A functional requirement is a statement of how a system must behave. It defines what the system should do in order to meet the user's needs or expectations. Functional requirements can be thought of as features that the user detects. They are different from non-functional requirements, which define how the system should work internally (e.g., performance, security, etc.).

Functional requirements are made up of two parts: function and behavior. The function is what the system does (e.g., "calculate sales tax").

3.4 Non Functional Requirements

Career recommendation non-functional requirements, like interests he has, how hours he can work likewise, with today's IT projects, to determine non-functional requirements, like availability, the approach requires that the designer 1st determine the scope: does the whole solution or only part of it need to be architected to meet minimum levels?

This is done through 4 steps:

- Identify the critical areas of solutions
- Identify the critical components within each critical area.
- Determine each components availability and risk.
- Model worst-case failure scenarios.

Non-Functional Requirements are the constraints or the requirements imposed on the system. They specify the quality attribute of the software. Non-Functional Requirements deal with issues like scalability, maintainability, performance, portability, security, reliability, and many more. Non-Functional Requirements address vital issues of quality for software systems.

Non-functional requirements (NFRs) are an essential part of software engineering. They describe

the characteristics and qualities of the software system that are not related to its functional requirements. These qualities include performance, reliability, maintainability, usability, security, and many others. Non-functional requirements are critical to the success of a software system as they ensure that it meets the expectations of its users and stakeholders.

Some common examples of non-functional requirements include:

Performance: Performance requirements describe the expected performance of the software system, such as its response time, throughput, and resource utilization. The software system should perform efficiently under different conditions and loads.

Reliability: Reliability requirements describe the ability of the software system to perform its functions consistently and accurately over time. The system should be available and responsive when needed, and should not experience frequent failures or crashes.

Usability: Usability requirements describe the ease of use and user-friendliness of the software system. The system should be easy to navigate and understand, and should provide clear and concise feedback to the user.

3.5 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

This includes evaluating the availability of suitable frameworks like Django for development, researching market demand and competition, estimating development and operational costs, ensuring legal compliance, analyzing operational impacts, and identifying potential risks and mitigation strategies. Engaging stakeholders and conducting thorough analyses enable informed decision-making regarding the project's feasibility, highlighting opportunities, challenges, and necessary adjustments to maximize success and minimize risks during implementation.

a. Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

Economic feasibility is a crucial aspect of a feasibility study for a cognitive agent system focused on code retrieval from repositories. It involves evaluating whether the project is financially viable and whether the benefits outweigh the costs. This includes assessing the initial investment required for development, implementation, and maintenance of the system, as well as estimating the potential returns or cost savings generated by the system over time. Factors such as cost-benefit analysis, return on investment (ROI), payback period, and net present value (NPV) are typically considered to determine the economic feasibility of the project. Additionally, economic feasibility involves analyzing factors such as revenue generation opportunities, cost reduction benefits, competitive advantage, and long-term sustainability to ensure that the project aligns with the organization's financial goals and objectives.

b. Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

Technical feasibility is a pivotal aspect in determining the viability of a cognitive agent system designed for code retrieval from repositories. It involves evaluating the availability of suitable technology and infrastructure, such as frameworks like Django, programming languages, and development tools needed for system implementation. Additionally, assessing the expertise of personnel in areas like natural language processing (NLP), machine learning, web development, and system integration is essential. The feasibility study also considers data integration capabilities with code repositories, API services, and AI algorithms, ensuring data security and compliance. Evaluating the system's performance, scalability, compatibility with existing systems, and potential technical risks and challenges provides stakeholders with valuable insights to make informed decisions regarding the project's technical feasibility and implementation strategy.

c. Operational Feasibility

Proposed projects are beneficial only if they can be turned out into information system. That will meet the organization’s operating requirements. Operational feasibility amount of fund that the company can pour into the research and development of the system is limited. The aspects of the project are to be taken as an important part of the project implementation.

Some of the important issues raised are to test the operational feasibility of a project includes the

following:-

- Is there sufficient support for the management from the users?
- Will the system be used and work properly if it is being developed and implemented?
- Will there be any resistance from the user that will undermine the possible application benefits?

This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration. So there is no questionofresistancefromtheusersthatcanunderminethepossibleapplicationbenefits.Thewell- planned design would ensure the optimal utilization of the computer resources.

4. SYSTEM REQUIREMENTS

4.1 Software Requirements

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation. The appropriation of requirements and implementation constraints gives the general overview of the project in regard to what the areas of strength and deficit are and how to tackle them.

- Operating system **:** Windows7 Ultimate
- Coding Language **:** Python
- Front-End **:** Python
- Designing **:** Html, CSS, JavaScript
- Data Base **:** My SQL

4.2 Hardware Requirements

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- System **:** Intel Core i5
- Hard Disk **:**120 GB
- Monitor **:** Dell inspiron15 3000
- Input Devices **:** Keyboard, Mouse
- Ram **:** 8 GB

5. SYSTEM DESIGN

5.1 System Architecture

The system design for implementing smart contracts in automated and transparent supply chain management in agriculture involves a block chain-based architecture that ensures secure, immutable, and transparent transactions among all stakeholders, including farmers, suppliers, distributors, retailers, and consumers. The system comprises a web/mobile application for user interaction, a block chain network such as Ethereum, Polygon, or Hyper ledger Fabric for decentralization, and smart contracts written in Solidity or Chain code to automate key processes like farm registration, product tracking, payment and escrow management, quality verification, and dispute resolution. Off-chain storage solutions like IPFS and cloud databases (PostgreSQL/MongoDB) are used to store large metadata and documents such as quality certificates and transaction records, while IoT sensors such as GPS, temperature, and humidity trackers ensure real-time monitoring of agricultural products throughout the supply chain. Oracles like Chainlink or custom APIs are integrated to fetch external data for smart contract execution, ensuring accurate and real-world information updates. The system workflow starts with user registration and KYC verification, followed by farmers logging crop details onto the blockchain, after which transportation logistics are monitored using IoT devices, and quality inspectors verify standards before data is uploaded onto the blockchain for validation. Once all predefined conditions are met, escrow-based smart contracts automatically release payments, enabling transparent and tamper- proof financial transactions. Consumers can trace the entire history of a product by scanning QR codes linked to blockchain records, ensuring trust and authenticity. The technology stack includes React.js or Flutter for frontend development, Node.js or FastAPI for backend services, Solidity for smart contract development, PostgreSQL or MongoDB for database management, IPFS or AWS S3 for decentralized file storage, and Raspberry Pi or Arduino for IoT integration. Security measures such as data encryption, smart contract audits, role-based access control (RBAC), and compliance with food safety regulations are implemented to ensure robust protection against fraud and cyber threats. This system ultimately enhances trust, reduces inefficiencies, automates payments, and provides end-to-end visibility in the agricultural supply chain, transforming traditional processes into a decentralized and tamper-proof ecosystem.

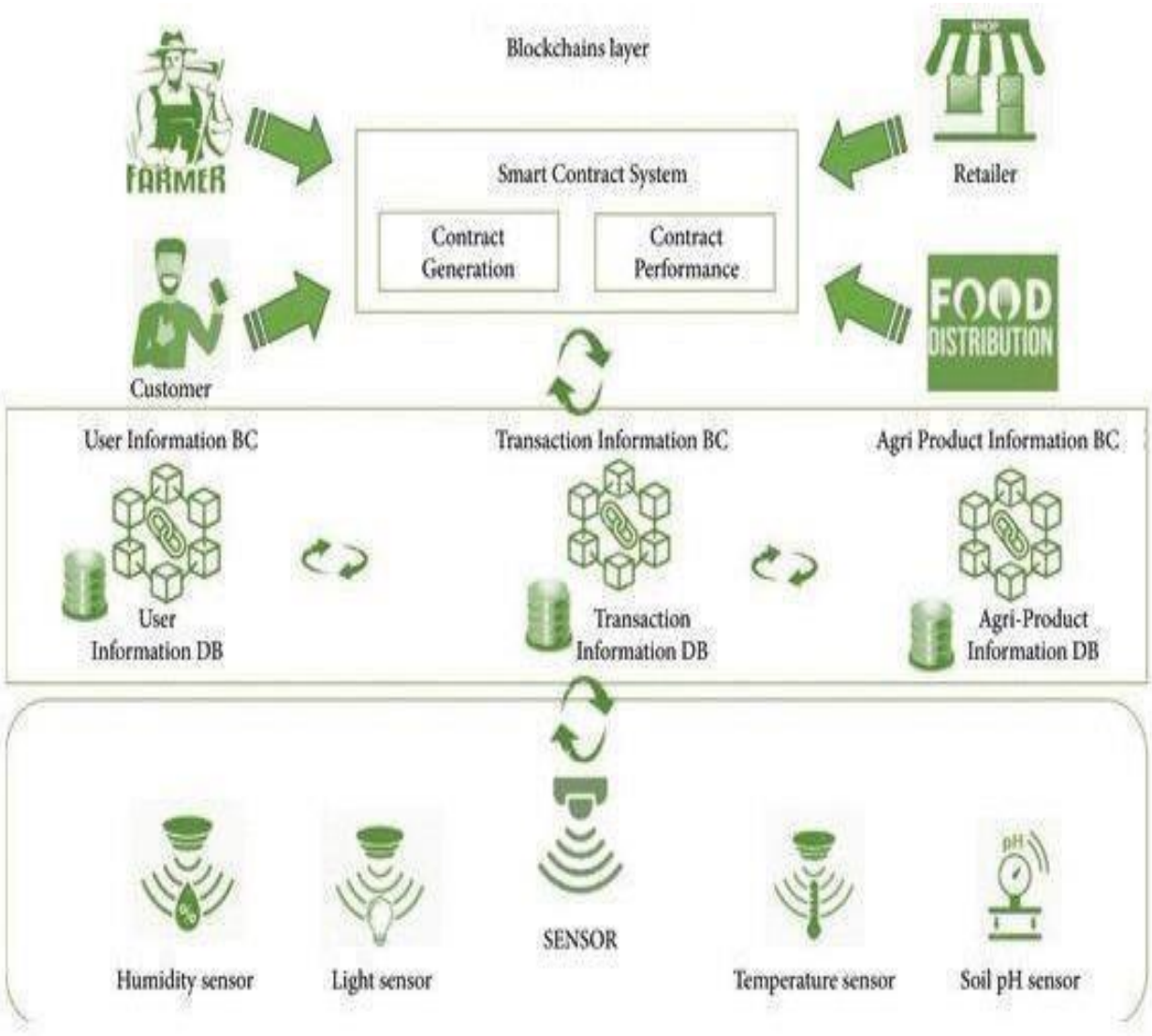


Fig no: 5.1.1 System Architecture

5.2 Data flow diagram

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. An eat and clear DFD can depict the right amount of the system requirement graphically. It may be used as a communication tool between a system analyst and any person who plays apart in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

5.3 UML Diagrams

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems. UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.

There are several types of UML diagrams and each one of them serves a different purpose regardless of whether it is being designed before the implementation or after (as part of documentation).

The UML class diagram for a cognitive agent system aimed at retrieving relevant code components from a repository would include several key classes interconnected to represent the system's functionality. At the center, the User Interface class facilitates user interactions, including methods like display results or presenting search outcomes and get user input to capture user queries. This class collaborates with the Query processor class, responsible for preprocessing and analyzing queries through methods such as preprocess query and analyze query.

Supporting classes like user input manage user input data, processed query stores processed query results, code component represents retrieved code snippets with attributes like code component and code content and relevant code stores relevance scores for the retrieved code components. The query class handles search queries, while the class manages search out comes in a list format. UML has a direct relation with object- oriented analysis and design. After some standardization, UML has become an OMG standard. The two broadest categories that encompass all other types are:

• Structural UML diagram.

• Behavioral UML diagram.

As the name suggests, some UML diagrams try to analyses and depict the structure of a system or process, where as other describe the behavior of the system, its actors, and its building components.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users already-to-use, expressive visual modeling Language
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frame works, patterns and components.
- Integrate best practices.

A semantic search engine enhances search accuracy by considering context and relationships between code components. The relevance determination module utilizes machine learning and ranking algorithms to assess and present the most relevant results to users. Features for user feedback, collaboration, performance optimization, security, and analytics contribute to a robust and user-friendly system that empowers developers in their coding endeavors.

Structural UML diagrams illustrate the organization of a system by depicting its components, such as classes, objects, and packages. They represent the elements that make up the system and the relationships between them.

5.3.1 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals.

Roles of the actors in the system can be depicted. The use case in a cognitive agent system for retrieving relevant code components from repositories serves as a foundational blueprint, guiding the system's development, implementation, and functionality. It outlines the actors involved, such as developers and the cognitive agent system, and defines their interactions and roles within the system. The use case drives the design and architecture of key components like the user interface, query processor, code repository integration, semantic search engine, and relevance determination module, ensuring that they align with the specified requirements for accurate and efficient code retrieval. Test cases derived from the use case validate the system's functionality, while user feedback mechanisms outlined in the use case contribute to continuous learning and improvement, ultimately delivering a system that meets the objectives of streamlined and relevant code component retrieval.

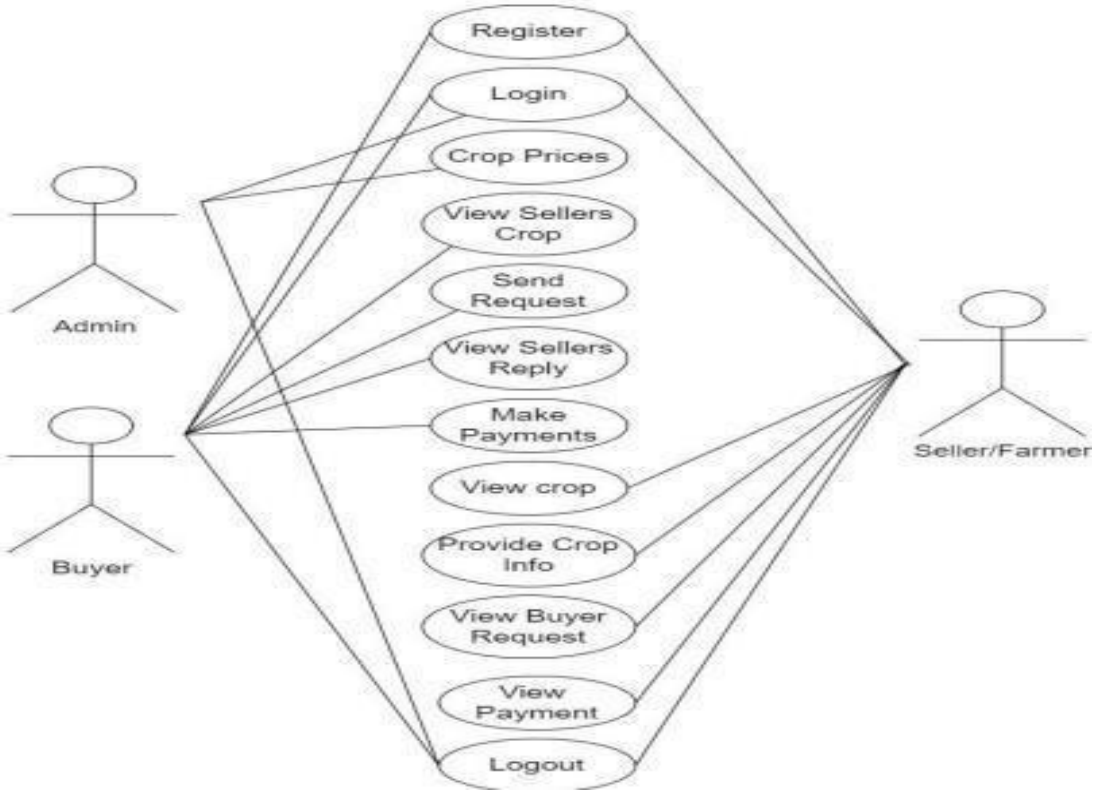


Fig no: 5.1.2 Use case Diagram

5.3.2 CLASS DIAGRAM:

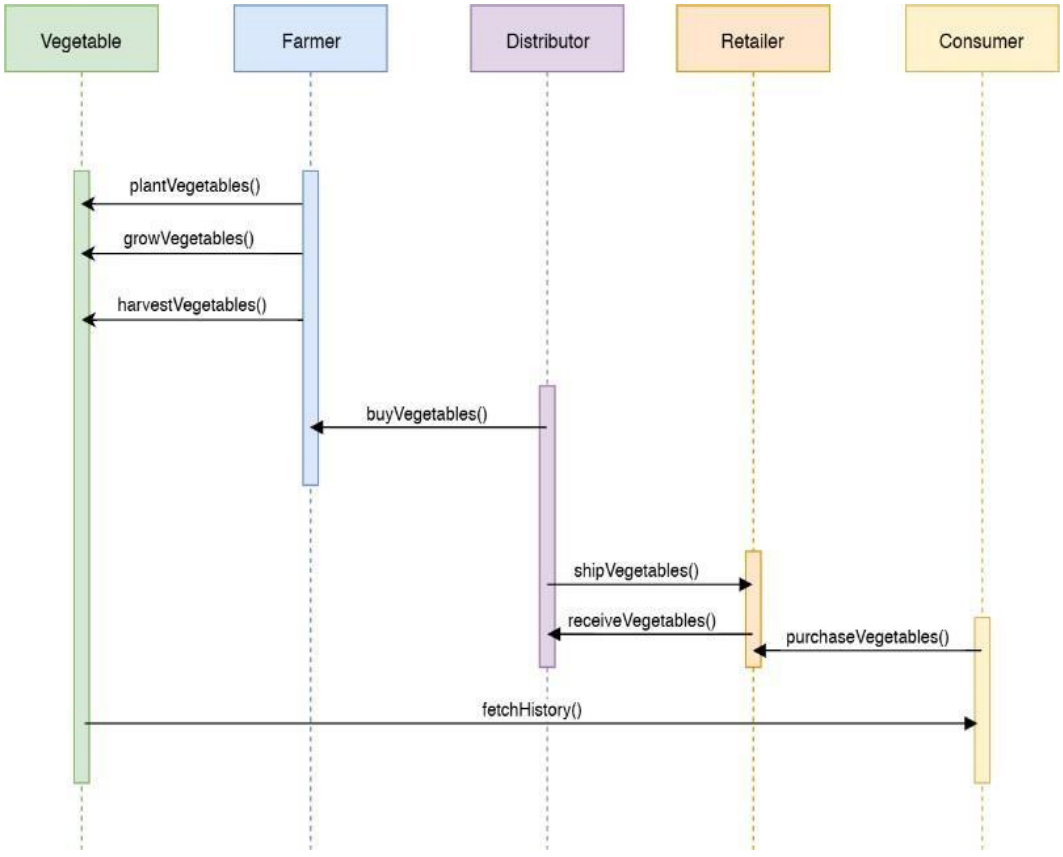
In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

Fig no: 5.1.3 Class Diagram

5.3.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

- A sequence diagram simply depicts the interaction between the objects in a sequential order i.e. the order in which these interactions occur.
- These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.
- Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration.



Figno:5.1.4 Sequence Diagram

5.3.4 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of work flows of step wise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step work flows of components in a system. An activity diagram shows the overall flow of control.

Activity Diagrams are used to illustrate the flow of control in a system and refer to the steps involved in the execution of a use case. We can depict both sequential processing and concurrent processing of activities using an activity diagram ie an activity diagram focuses on the condition of flow and the sequence in which it happens.

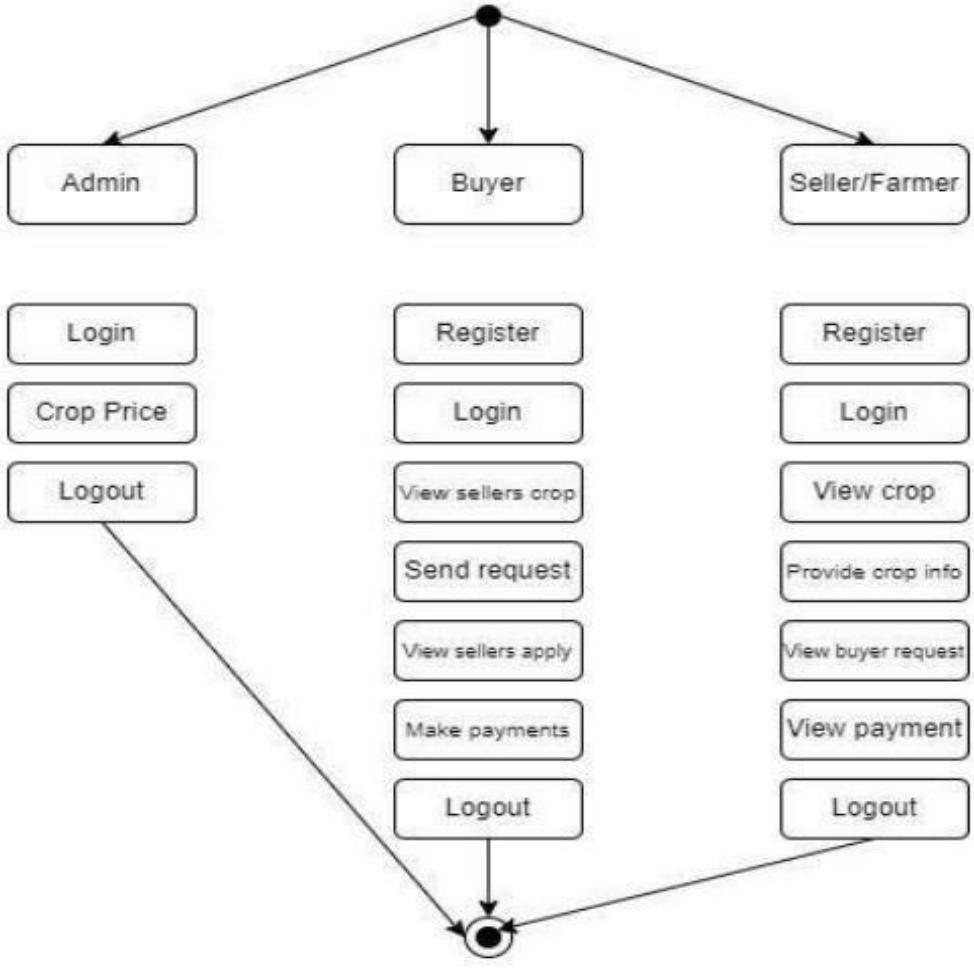


Fig no: 5.1.5 Activity Diagram

6. CODING AND ITS IMPLEMENTATION

6.1 SOURCE CODE

```
from django.shortcuts import render
from django.template import RequestContext
from django.contrib import messages
from django.http import HttpResponse
from django.core.files.storage import FileSystemStorage
import os
from datetime import date
import json
from web3 import Web3, HTTPProvider
import pickle

global username
global contract, web3, product_name
global usersList, cropList

#function to call contract
def getContract():
    global contract, web3
    blockchain_address = 'http://127.0.0.1:9545'
    web3 = Web3(HTTPProvider(blockchain_address))
    web3.eth.defaultAccount = web3.eth.accounts[0]
    compiled_contract_path = 'Agricultural.json' #Agricultural contract file
    deployed_contract_address = '0x7e419A8740a79F3E831Ba14519DEEC96ce932D32' #contract address
    with open(compiled_contract_path) as file:
        contract_json = json.load(file)  # load contract info as JSON
        contract_abi = contract_json['abi']  # fetch contract's abi - necessary to call its functions
    file.close()
    contract = web3.eth.contract(address=deployed_contract_address, abi=contract_abi)
getContract()
```

```
def getUsersList():
    global usersList, contract
    usersList = []
    count = contract.functions.getUserCount().call()
    print(str(count)+"======================================")
    for i in range(0, count):
        user = contract.functions.getUsername(i).call()
        password = contract.functions.getPassword(i).call()
        email = contract.functions.getEmail(i).call()
        utype = contract.functions.getType(i).call()
        usersList.append([user, password, email, utype])

def getCropList():
    global cropList, contract
    cropList = []
    count = contract.functions.getCropCount().call()
    for i in range(0, count):
        owner = contract.functions.getOwner(i).call()
        cid = contract.functions.getCropid(i).call()
        cname = contract.functions.getCropname(i).call()
        qty = contract.functions.getQuantity(i).call()
        price = contract.functions.getPrice(i).call()
        category = contract.functions.getCategory(i).call()
        sold = contract.functions.getSold(i).call()
        yield_date = contract.functions.getDate(i).call()
        buyer = contract.functions.getBuyer(i).call()
        cropList.append([owner, cid, cname, qty, price, category, sold, yield_date, buyer])
getUsersList()
getCropList()

def PurchaseAction(request):
    if request.method == 'POST':
        global username, cropList
        row = request.POST.get('t1', False)
```

```
        qty = request.POST.get('t2', False)
        card = request.POST.get('t3', False)
        plist = cropList[int(row)]
        data = plist[8]
        output = ""
        if data == "-":
            output = username+","+qty+","+str(float(plist[4]) * float(qty))+","+card+",-$"
        else:
            output += data+username+","+qty+","+str(float(plist[4]) * float(qty))+","+card+",-$"
        plist[8] = output
        contract.functions.updateBuyer(int(row), qty, output).transact()
        context= {'data':"Your Buy Request Successfully Sent to "+plist[0]}
        return render(request, 'BuyerScreen.html', context)

def Purchase(request):
    if request.method == 'GET':
        global cropList
        row = request.GET['t1']
        output = '<tr><td><font size="3" color="black">Crop ID</td>'
            output += '<td><input type="text" name="t1" style="font-family: Comic Sans MS" size="15" value="'+row+'" readonly></td></tr>'
        context= {'data1':output}
        return render(request, 'Purchase.html', context)

def SellerCrop(request):
    if request.method == 'GET':
        global cropList, username
        output = '<table border=1 align=center>'
        output+='<tr><th><font size=3 color=black>Farmer/Seller</font></th>'
        output+='<th><font size=3 color=black>Crop Id</font></th>'
        output+='<th><font size=3 color=black>Crop Name</font></th>'
        output+='<th><font size=3 color=black>Category</font></th>'
        output+='<th><font size=3 color=black>Quantity</font></th>'
        output+='<th><font size=3 color=black>Price</font></th>'
```

```
        output+='<th><font size=3 color=black>Available Quantity</font></th>'
        output+='<th><font size=3 color=black>Expected Yield Date</font></th>'
        output+='<th><font size=3 color=black>Crop Image</font></th>'
        output+='<th><font size=3 color=black>Purchase Crop</font></th></tr>'
        for i in range(len(cropList)):
            plist = cropList[i]
            output+='<tr><td><font size=3 color=black>'+plist[0]+'</font></td>'
            output+='<td><font size=3 color=black>'+plist[1]+'</font></td>'
            output+='<td><font size=3 color=black>'+str(plist[2])+'</font></td>'
            output+='<td><font size=3 color=black>'+str(plist[5])+'</font></td>'
            output+='<td><font size=3 color=black>'+str(plist[3])+'</font></td>'
            output+='<td><font size=3 color=black>'+str(plist[4])+'</font></td>'
            output+='<td><font size=3 color=black>'+str(float(plist[3]) - float(plist[6]))+'</font></td>'
            output+='<td><font size=3 color=black>'+str(plist[7])+'</font></td>'
            output+='<td><imgsrc="/static/files/'+plist[1]+'.png" width="200" height="200"></img></td>'
                    output+='<td><a href=\'Purchase?t1='+str(i)+'\'><font size=3 color=green>Purchased
Crop</font></a></td></tr>'
        output+="</table><br/><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'BuyerScreen.html', context)

def ViewReply(request):
    if request.method == 'GET':
        global cropList, username
        output = '<table border=1 align=center>'
        output+='<tr><th><font size=3 color=black>Farmer/Seller</font></th>'
        output+='<th><font size=3 color=black>Crop Id</font></th>'
        output+='<th><font size=3 color=black>Crop Name</font></th>'
        output+='<th><font size=3 color=black>Available Quantity</font></th>'
        output+='<th><font size=3 color=black>Buyer Name</font></th>'
        output+='<th><font size=3 color=black>Requested Quantity</font></th>'
        output+='<th><font size=3 color=black>Total Amount</font></th>'
        output+='<th><font size=3 color=black>Card Name</font></th>'
        output+='<th><font size=3 color=black>Purchase Status</font></th></tr>'
```

```
        for i in range(len(cropList)):
            plist = cropList[i]
            buyer = plist[8]
            if buyer != '-':
                arr = buyer.split("$")
                for j in range(len(arr)-1):
                    data = arr[j].split(",")
                    print(data)
                    if data[0] == username:
                        output+='<tr><td><font size=3 color=black>'+plist[0]+'</font></td>'
                        output+='<td><font size=3 color=black>'+plist[1]+'</font></td>'
                        output+='<td><font size=3 color=black>'+str(plist[2])+'</font></td>'
                        output+='<td><font size=3 color=black>'+str(float(plist[3]) - float(plist[6]))+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[0]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[1]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[2]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[3]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[4]+'</font></td></tr>'
        output+="</table><br/><br/><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'BuyerScreen.html', context)

def ViewCropDetails(request):
    if request.method == 'GET':
        global cropList, username
        output = '<table border=1 align=center>'
        output+='<tr><th><font size=3 color=black>Farmer/Seller</font></th>'
        output+='<th><font size=3 color=black>Crop Id</font></th>'
        output+='<th><font size=3 color=black>Crop Name</font></th>'
        output+='<th><font size=3 color=black>Category</font></th>'
        output+='<th><font size=3 color=black>Quantity</font></th>'
        output+='<th><font size=3 color=black>Price</font></th>'
        output+='<th><font size=3 color=black>Available Quantity</font></th>'
        output+='<th><font size=3 color=black>Expected Yield Date</font></th>'
```

```
        output+='<th><font size=3 color=black>Crop Image</font></th></tr>'
        for i in range(len(cropList)):
            plist = cropList[i]
            if plist[0] == username:
                output+='<tr><td><font size=3 color=black>'+plist[0]+'</font></td>'
                output+='<td><font size=3 color=black>'+plist[1]+'</font></td>'
                output+='<td><font size=3 color=black>'+str(plist[2])+'</font></td>'
                output+='<td><font size=3 color=black>'+str(plist[5])+'</font></td>'
                output+='<td><font size=3 color=black>'+str(plist[3])+'</font></td>'
                output+='<td><font size=3 color=black>'+str(plist[4])+'</font></td>'
                output+='<td><font size=3 color=black>'+str(float(plist[3]) - float(plist[6]))+'</font></td>'
                output+='<td><font size=3 color=black>'+str(plist[7])+'</font></td>'
                                        output+='<td><imgsrc="/static/files/'+plist[1]+'.png"        width="200"
height="200"></img></td></tr>'
        output+="</table><br/><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'SellerScreen.html', context)

def ViewPayments(request):
    if request.method == 'GET':
        global cropList, username
        output = '<table border=1 align=center>'
        output+='<tr><th><font size=3 color=black>Farmer/Seller</font></th>'
        output+='<th><font size=3 color=black>Crop Id</font></th>'
        output+='<th><font size=3 color=black>Crop Name</font></th>'
        output+='<th><font size=3 color=black>Available Quantity</font></th>'
        output+='<th><font size=3 color=black>Buyer Name</font></th>'
        output+='<th><font size=3 color=black>Requested Quantity</font></th>'
        output+='<th><font size=3 color=black>Total Amount</font></th>'
        output+='<th><font size=3 color=black>Card Name</font></th>'
        output+='<th><font size=3 color=black>Purchase Status</font></th></tr>'
        for i in range(len(cropList)):
            plist = cropList[i]
            if plist[0] == username:
```

```
            buyer = plist[8]
            if buyer != '-':
                arr = buyer.split("$")
                for j in range(len(arr)-1):
                    data = arr[j].split(",")
                    if data[4] != '-':
                        output+='<tr><td><font size=3 color=black>'+plist[0]+'</font></td>'
                        output+='<td><font size=3 color=black>'+plist[1]+'</font></td>'
                        output+='<td><font size=3 color=black>'+str(plist[2])+'</font></td>'
                        output+='<td><font size=3 color=black>'+str(float(plist[3]) - float(plist[6]))+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[0]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[1]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[2]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[3]+'</font></td>'
                        output+='<td><font size=3 color=black>'+data[4]+'</font></td></tr>'
        output+="</table><br/><br/><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'SellerScreen.html', context)

def ConfirmPurchase(request):
    if request.method == 'GET':
        global cropList
        row = request.GET['t1']
        col = request.GET['t2']
        status = request.GET['t3']
        plist = cropList[int(row)]
        arr = plist[8].split("$")
        data = arr[int(col)].split(",")
        output = ""
        for i in range(len(arr)-1):
            if i != int(col):
                output += arr[i]+"$"
        output += data[0]+","+data[1]+","+data[2]+","+data[3]+","+status+"$"
        plist[8] = output
```

```
        plist[6] = float(plist[6]) + float(data[1])
        contract.functions.updateBuyer(int(row), str(float(plist[6]) + float(data[1])), output).transact()
        context= {'data':"Buyer Request Successfully "+status}
        return render(request, 'SellerScreen.html', context)

def ViewBuyerRequest(request):
    if request.method == 'GET':
        global cropList, username
        output = '<table border=1 align=center>'
        output+='<tr><th><font size=3 color=black>Farmer/Seller</font></th>'
        output+='<th><font size=3 color=black>Crop Id</font></th>'
        output+='<th><font size=3 color=black>Crop Name</font></th>'
        output+='<th><font size=3 color=black>Available Quantity</font></th>'
        output+='<th><font size=3 color=black>Buyer Name</font></th>'
        output+='<th><font size=3 color=black>Requested Quantity</font></th>'
        output+='<th><font size=3 color=black>Total Amount</font></th>'
        output+='<th><font size=3 color=black>Accept Request</font></th>'
        output+='<th><font size=3 color=black>Reject Request</font></th></tr>'
        for i in range(len(cropList)):
            plist = cropList[i]
            if plist[0] == username:
                buyer = plist[8]
                if buyer != '-':
                    arr = buyer.split("$")
                    for j in range(len(arr)-1):
                        data = arr[j].split(",")
                        if data[4] == '-':
                            output+='<tr><td><font size=3 color=black>'+plist[0]+'</font></td>'
                            output+='<td><font size=3 color=black>'+plist[1]+'</font></td>'
                            output+='<td><font size=3 color=black>'+str(plist[2])+'</font></td>'
                            output+='<td><font size=3 color=black>'+str(float(plist[3]) - float(plist[6]))+'</font></td>'
                            output+='<td><font size=3 color=black>'+data[0]+'</font></td>'
                            output+='<td><font size=3 color=black>'+data[1]+'</font></td>'
                            output+='<td><font size=3 color=black>'+data[2]+'</font></td>'
```

```
                    output+='<td><a href=\'ConfirmPurchase?t1='+str(i)+'&t2='+str(j)+'&t3=Accepted\'><font size=3 color=green>Accept</font></a></td>'
                    output+='<td><a href=\'ConfirmPurchase?t1='+str(i)+'&t2='+str(j)+'&t3=Rejected\'><font size=3 color=red>Reject</font></a></td></tr>'
        output+="</table><br/><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'SellerScreen.html', context)

def index(request):
    if request.method == 'GET':
        return render(request, 'index.html', {})

def BuyerLogin(request):
    if request.method == 'GET':
        return render(request, 'BuyerLogin.html', {})

def SellerLogin(request):
    if request.method == 'GET':
        return render(request, 'SellerLogin.html', {})

def Register(request):
    if request.method == 'GET':
        return render(request, 'Register.html', {})

def UploadCropInfo(request):
    if request.method == 'GET':
        return render(request, 'UploadCropInfo.html', {})

def UploadCropInfoAction(request):
    if request.method == 'POST':
        global username, cropList
        cname = request.POST.get('t1', False)
        category = request.POST.get('t2', False)
        qty = request.POST.get('t3', False)
```

```
        price = request.POST.get('t4', False)
        yield_date = request.POST.get('t5', False)
        image = request.FILES['t6']
        imagename = request.FILES['t6'].name
        cid = str(len(cropList) + 1)

        fs = FileSystemStorage()
         msg = contract.functions.createCrop(username, str(cid), cname, qty, price, category, "0", yield_date, '-
').transact()
        tx_receipt = web3.eth.waitForTransactionReceipt(msg)
        if os.path.exists('AgricultureApp/static/files/'+str(cid)+".png"):
            os.remove('AgricultureApp/static/files/'+str(cid)+".png")
        filename = fs.save('AgricultureApp/static/files/'+str(cid)+".png", image)
        cropList.append([username, str(cid), cname, qty, price, category, "0", yield_date, '-'])
        context= {'data':"Crop details saved in Blockchain with Crop ID = "+cid+"<br/><br/>"+str(tx_receipt)}
        return render(request, 'UploadCropInfo.html', context)


def RegisterAction(request):
    if request.method == 'POST':
        global usersList
        username = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        contact = request.POST.get('t3', False)
        email = request.POST.get('t4', False)
        address = request.POST.get('t5', False)
        utype = request.POST.get('t6', False)
        count = contract.functions.getUserCount().call()
        status = "none"
        for i in range(0, count):
            user1 = contract.functions.getUsername(i).call()
            if username == user1:
                status = "exists"
                break
```

```
        if status == "none":
            msg = contract.functions.createUser(username, email, password, contact, address, utype).transact()
            tx_receipt = web3.eth.waitForTransactionReceipt(msg)
            usersList.append([username, password, email, utype])
            context= {'data':'Signup Process Completed<br/>'+str(tx_receipt)}
            return render(request, 'Register.html', context)
        else:
            context= {'data':'Given username already exists'}
            return render(request, 'Register.html', context)

def BuyerLoginAction(request):
    if request.method == 'POST':
        global username, contract, usersList
        uname = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        status = "BuyerLogin.html"
        output = 'Invalid login details'
        for i in range(len(usersList)):
            ulist = usersList[i]
            if  ulist[0] == uname and ulist[1] == password and ulist[3] == "Buyer":
                username = uname
                status = "BuyerScreen.html"
                output = 'Welcome '+username
                break
        context= {'data':output}
        return render(request, status, context)

def SellerLoginAction(request):
    if request.method == 'POST':
        global username, contract, usersList
        uname = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        status = "SellerLogin.html"
        output = 'Invalid login details'
```

```
for i in range(len(usersList)):
    ulist = usersList[i]
    if  ulist[0] == uname and ulist[1] == password and ulist[3] == "Seller":
        username = uname
        status = "SellerScreen.html"
        output = 'Welcome '+username
        break
context= {'data':output}
return render(request, status, context)
```

6.2 Implementation

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using white space indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

What is Python

Python is a popular programming language. It was created by Guido van Rossum, and release in 1991.

It is used for:

- Web development (server-side),
- Software development,
- mathematics,
- system scripting.

What can Python do

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to data base systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python

- Python works on different platforms like Windows, Mac, Linux, and Raspberry Pi. Python has the simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.

Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial. However, Python 2, although not being updated with anything other than security updates, is still quite popular.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with Influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using white space, to define scope; such as the scope of loops,functions and classes. Other programming languages often use curly-brackets for this purpose.

How python used:

In a cognitive agent system, Python can be used in several ways to retrieve relevant code components from a repository. A cognitive agent system typically involves intelligent agents that can reason, learn, and interact with their environment. Python, with its versatile libraries and frameworks, can play a crucial role in implementing such systems. Here's how Python can be utilized:

Version Control System Integration:

Python can be used to integrate with version control systems like Git or SVN. Libraries such as Git Python or PyGit huballow agents to interact with repositories, clone code bases, retrieve commit history, and manage branches.

Code Search Analysis:

Python's natural language processing (NLP) capabilities through libraries like NLTKorspaCy can be used to analyze code comments, documentation, and commit messages. This analysis helps in understanding the context and relevance of code components.

```
From nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
code_comment="This function calculates the factorial of a number."
tokens = word_tokenize(code_comment)
filtered_tokens=[tokenfortoken in tokensiftoken.lower() notin stopwords.words('english')]
```

Machine Learning For Code Retrieval:

Python's extensive machine learning libraries, such as sci kit-learnor Tensor Flow, can be employed to build models that learn from past interactions and user preferences. These models can suggest relevant code components based on similarity metrics or user behavior.

```
From sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
documents = ["Calculate the factorial of a number.",
        "Sort a list of integers in ascending order.", "Compute
        the sum of elements in an array."]
vectorizer=TfidfVectorizer()
```

```
tfidf_matrix = vectorizer.fit_transform(documents)

similarity_matrix = cosine_similarity(tfidf_matrix,tfidf_matrix)
```

Semantic Code Search Engines:

Python can be used to develop the semantic code search engines using tools like Code BERT or the Code Search Net. These engines leverage pertained language models to understand code semantics and retrieve relevant code snippets efficiently.

```
From transformers import pipeline

code_search=pipeline("code-search",model="microsoft/codebert-base")

search_results = code_search("Calculate the factorial of a number")
```

API Integration:

Python can interact with code hosting platforms like GitHub, GitLab, or Bitbucket using their APIs. This allows agents to search for code, retrieve metadata, and even contribute back to repositories programmatically.

```
import requests

url='https://api.github.com/search/code?q=factorial+language:python'

response = requests.get(url)

data=response.json()
```

By combining these approaches, a cognitive agent system can effectively retrieve relevant code components from repositories based on user queries, contextual information, and learning from past interactions.

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTINGS USED:

In a cognitive agent system designed to retrieve relevant code components from a repository, several types of testing can be applied to ensure the system's functionality and effectiveness. Here are some of the key types of testing that are commonly used:

Unit Testing:

Purpose: Unit testing is used to test individual units or components of the cognitive agent system in isolation. This ensures that each unit functions correctly and meets its specifications.

Testing Scenario: Unit tests can be written to validate functions or classes responsible for querying the code repository, processing retrieved code components, and handling data structures used in relevance determination.

Integration Testing:

Purpose: Integration testing verifies that different components of the cognitive agent system work together seamlessly. It ensures that interactions between modules, such as the retrieval module and the relevance determination module, function as intended.

Testing Scenario: Integration tests can simulate the flow of data and operations between modules, including interactions with the code repository, to validate the system's overall behavior.

Functional Testing:

Purpose: Functional testing evaluates the system's functionality against specified requirements. It ensures that the cognitive agent system performs the intended tasks accurately and effectively.

Testing Scenario: Functional tests for code retrieval would involve scenarios such as querying the repository with different search terms, verifying the correctness of retrieved code components, and assessing the relevance of the results.

Performance Testing:

Purpose: Performance testing assesses the system's responsiveness, scalability, and resource usage under varying workloads. It ensures that the system can handle the expected load efficiently.

Testing Scenario: Performance tests can be conducted to measure the response time of code retrieval operations, assess the system's ability to handle concurrent requests, and optimize resource utilization.

Usability Testing:

Purpose: Usability testing evaluates the user interface and user experience aspects of the cognitive agent system. It ensures that users can interact with the system easily and intuitively.

Testing Scenario: Usability tests can focus on the user interface for entering search queries, displaying retrieved code components, and providing relevant feedback to users during the retrieval process.

Security Testing:

Purpose: Security testing identifies and addresses potential security vulnerabilities in the cognitive agent system. It ensures that sensitive data and functionalities are protected from unauthorized access and attacks.

Testing Scenario: Security tests can assess the system's handling of authentication, authorization, data encryption (if applicable), and protection against common security threats such as injection attacks.

White Box Testing:

White-box testing can be used in the development and testing of a cognitive agent system designed to retrieve relevant code components from a repository. White-box testing, also known as structural testing or glass-box testing, involves examining the internal structure and logic of the software system. It is often used to ensure thorough coverage of code paths, identify logical errors, and validate the correctness of algorithms and decision-making processes within the system.

In the context of a cognitive agent system for code retrieval, white-box testing can be applied in several ways:

a. code coverage analysis

b. algorithm validation

c. error handling and exception testing

d. boundary value analysis

e. code review and inspection

Black Box Testing:

Black-box testing can also be used in the development and testing of a cognitive agent system designed to retrieve relevant code components from a repository. Black-box testing, also known as behavioral testing focuses on testing the system's functionality without considering its internal structure, code implementation, or logic. It evaluates the system's behavior based on inputs and expected outputs, simulating how end-users would interact with the system.

In the context of a cognitive agent system for code retrieval, black-box testing can be applied in several ways:

a. Boundary value analysis

b. Error handling testing

c. Regression testing

The specific testing approach and tools used in a project depend on the project's requirements, the team's expertise, and the development and testing strategies adopted by the organization. It's common for projects to use a combination of these testing types to ensure comprehensive testing coverage and software quality.

a. Boundary Value Analysis:

Boundary Value Analysis is based on testing the boundary values of valid and invalid partitions. The behavior at the edge of the equivalence partition is more likely to be incorrect than the behavior within the partition, so boundaries are an area where testing is likely to yield defects.

b. Error Handling Testing:

Error handling testing is a type of software testing that is performed to check whether the system is capable of or able to handle the errors that may happen in future. This type of testing is basically performed with the help of both developers and the testers. Error handling testing not only focuses on the determination of error but also focuses on the exception handling.

c. Regression Testing:

Regression testing is a type of software testing that ensures recent code changes have not adversely affected existing features. It involves retesting the software after modifications to confirm that previously developed and tested functionality still performs correctly. This process helps identify bugs that might have been introduced into existing areas of a software application due to new changes or enhancements.

7.2. TEST CASES

S.no	Test Case	Excepted Result	Result	Remarks (IF Fails)
1.	Verify smart contract deployment on the block chain network.	Smart contract is successfully deployed and available for execution.	Pass	Executed successfully
2.	Test **farmer registration** in the system using block chain.	Farmer's details (name, location, crop details) are stored on the block chain, ensuring immutability.	Pass	Algorithm implemented
3.	Validate **supply chain traceability** from farmer to consumer.	Each stage (harvesting, processing, storage, transportation) is logged on the block chain with timestamps.	Pass	Input uploaded successfully
4.	Ensure **automatic payment** via smart contracts.	When predefined conditions (e.g., delivery confirmation) are met, the smart contract releases payment to the farmer.	Fail	Cannot be executed
5.	Test **fraud prevention** by verifying data integrity.	If an unauthorized party attempts to alter a transaction, the block chain rejects the modification.	Fail	Fraud is not detected
6.	Verify **product recall mechanism** in case of contamination detection.	Smart contract triggers alerts to all stakeholders and prevents further distribution of contaminated batches.	Pass	Executed successfully
7.	Test **multi-party consensus** for transactions.	Transactions (e.g., quality checks, shipment approvals) proceed only when required parties approve the action.	Pass	Algorithm implemented
8.	Check **IoT sensor integration** with block chain for real-time monitoring (e.g., temperature, humidity).	Smart contract records IoT sensor data, ensuring storage and transport conditions meet safety standards.	Pass	Input uploaded successfully
9.	Validate **consumer access** to product history using block chain.	Consumers can scan a QR code and retrieve full product history, including origin, quality certification, and transport details.	Fail	Cannot be executed

10.	Ensure **penalty enforcement for contract violations** (e.g., late delivery, quality issues).	Smart contract applies penalties (e.g., fine deduction, refund processing) when violations occur.	Fail	Fraud is not detected

Table no: 7.2.1 Test Cases

8. OUTPUT SCREENS

Fig no. 8.1 new user sign up screen

Fig no. 8.2 User signing up successfully

Fig no. 8.3 Seller login screen

Fig no. 8.4 Seller welcome screen

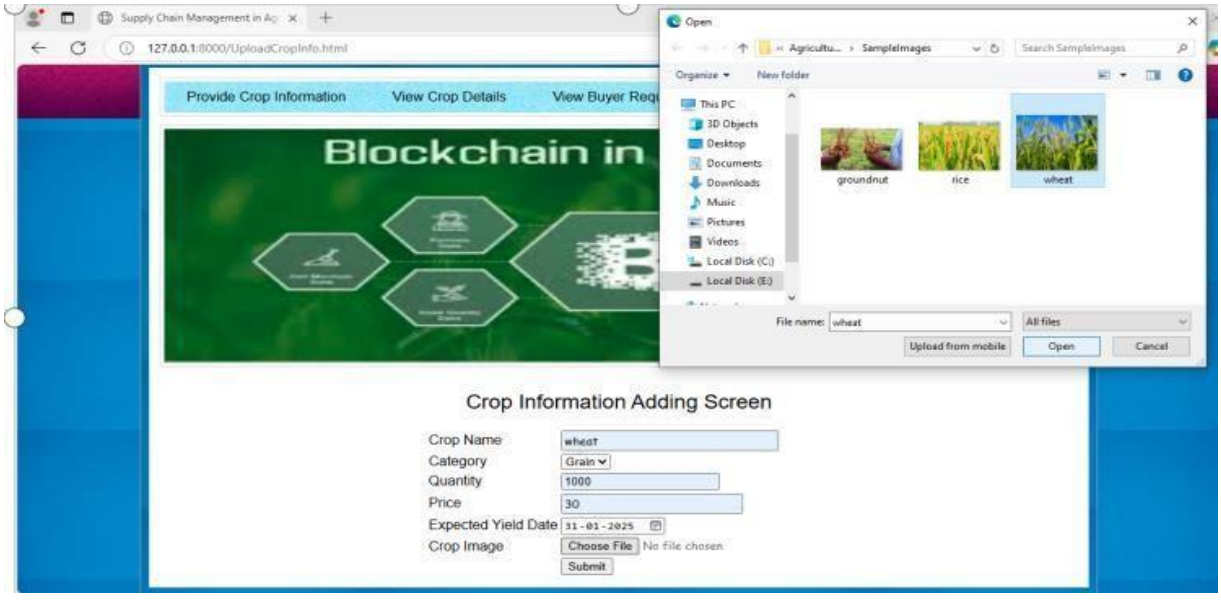


Fig no. 8.5 Crop adding screen

Farmer/Seller	Crop Id	Crop Name	Category	Quantity	Price	Available Quantity	Expected Yield Date	Crop Image
kumar	1	wheat	Grain	1000	30	1000.0	2025-01-31	
kumar	2	rice	Grain	2000	4000	2000.0	2025-01-21	

Fig no. 8.6 Crop availability

Fig no. 8.7 Buyer login screen

Fig no. 8.8 Buyer welcome screen

Farmer/Seller	Crop Id	Crop Name	Category	Quantity	Price	Available Quantity	Expected Yield Date	Crop Image	Purchase Crop
kumar	1	wheat	Grain	1000	30	1000.0	2025-01-31		Purchased Crop
kumar	2	rice	Grain	2000	4000	2000.0	2025-01-21		Purchased Crop

Fig no. 8.9 List of crop details

Fig no. 8.10 Crop purchase screen

Farmer/Seller	Crop Id	Crop Name	Available Quantity	Buyer Name	Requested Quantity	Total Amount	Card Name	Purchase Status
kumar	1	wheat	1000.0	rajesh	200	6000.0	SBI	-

Fig no.8.11 Payment gateway to purchase

Farmer/Seller	Crop Id	Crop Name	Available Quantity	Buyer Name	Requested Quantity	Total Amount	Accept Request	Reject Request
kumar	1	wheat	1000.0	rajesh	200	6000.0	Accept	Reject

Fig no. 8.12 Request acceptance

9. CONCLUSION

The implementation of smart contracts for automated and transparent supply chain management in agriculture has proven to be a highly effective approach to enhancing the efficiency, transparency, and accountability of the agricultural supply chain. Through the integration of blockchain technology, smart contracts enable real-time tracking, ensuring that every transaction and movement of goods is securely recorded on an immutable ledger, which fosters trust among stakeholders by providing access to transparent and reliable data. The automation of key processes such as contract execution, payments, and documentation has streamlined operations, reducing the need for intermediaries, cutting down administrative costs, and minimizing human error. As a result, the system has shown significant improvements in operational efficiency, with faster transaction times and more accurate inventory management. Furthermore, by ensuring that each party adheres to predefined conditions, smart contracts have helped build stronger relationships and greater confidence between farmers, suppliers, distributors, and retailers. Although challenges such as limited internet access in rural areas and digital literacy barriers remain, the project has demonstrated that the scalability of smart contracts can be adapted to different levels of the agricultural supply chain, from small farms to large-scale operations. Ultimately, this project has laid the foundation for a more automated, secure, and cost-effective agricultural supply chain, with the potential to improve global food security and sustainability. Going forward, continued refinement of the technology and overcoming accessibility challenges will be key to achieving wider adoption and maximizing the benefits of smart contracts across diverse agricultural settings. The implementation of smart contracts for automated and transparent supply chain management in agriculture has shown immense potential in transforming traditional agricultural systems by leveraging blockchain technology. Smart contracts facilitate the automation of key processes like payments, contract execution, and documentation, which significantly reduces human intervention, lowers the chances of errors, and improves the overall speed of operations. By embedding predefined rules and conditions into the system, these contracts ensure that once agreed terms are met, transactions occur seamlessly, which helps reduce delays and friction in the supply chain.

10. FUTURE ENHANCEMENTS

Looking forward, several enhancements could further optimize the use of smart contracts in agricultural supply chain management, addressing current challenges and expanding the system's capabilities. One critical area for improvement is the integration of **IoT (Internet of Things) devices** with smart contracts. By connecting sensors and devices on farms, warehouses, and transport vehicles, real-time data on environmental conditions, product quality, location, and inventory levels could automatically trigger contract clauses, further streamlining operations. For example, a temperature sensor in a transport vehicle could automatically trigger a payment release upon confirming that the cargo was kept within the required temperature range, ensuring the quality of perishable goods.

Additionally, expanding the system's **interoperability** is crucial to ensure that smart contracts can work seamlessly across different platforms, software, and blockchain networks. Standardized protocols could allow for better data exchange between stakeholders, from local farmers to global suppliers, enhancing collaboration and reducing friction in the supply chain. Moreover, **user-friendly interfaces** and **training programs** will be essential for increasing adoption among small farmers and those in rural areas, many of whom may lack technical expertise. Simplifying the technology and providing education on its use will be key to ensuring that everyone in the supply chain can benefit from the system.

A further enhancement could involve integrating **AI (Artificial Intelligence)** and **machine learning** algorithms to make smarter, predictive decisions within the supply chain. For instance, AI could analyze trends in weather patterns, crop yields, and market demands, automatically adjusting contracts based on predicted conditions. Finally, the **sustainability aspect** of smart contracts can be further enhanced by incorporating environmental impact metrics, such as carbon footprints or water usage. Smart contracts could incentivize farmers to adopt more sustainable practices. In conclusion, future enhancements to smart contract-based systems in agricultural supply chains will focus on improving automation, increasing accessibility, integrating AI, scaling blockchain solutions, and aligning with sustainability goals. These innovations will unlock greater value for farmers, suppliers, consumers, and the planet, ultimately leading to a more efficient, transparent, and sustainable global agricultural industry.

11. REFERENCES

1) X. Wang and X. Bi, "Application of BIM Algorithm and Block Chain Technology in the Construction of Agricultural Safety Traceability System,"2024International Conference on Distributed Computing and Optimization Techniques (ICDCOT), Bengaluru, India, 2024. https://ieeexplore.ieee.org/document/10515791

2) M. Musthafa Sheriff and D. John Aravindhar, "Integrated Block chain-Based Agri-Food Traceability and Deep Learning for Profit-Optimized Supply Chain Management in Agri-Food Supply Chains, "Bhilai, India, 2024.
https://ieeexplore.ieee.org/document/10480814

3) A. Toke, M. F. Monir and T. Ahmed, "Block chain in Agriculture: A Scalable Solution for Crop Quality Assessment and Data Integrity," 2024 IEEE International Black Sea Conference on Communications and Networking (Black Sea Com), Tbilisi, Georgia, 2024. https://ieeexplore.ieee.org/document/10646243

4) L. Ma, F. Gaoand X. Li, "Harnessing Block chain for Agricultural Product Traceability," 2024 4th International Conference on Computer Science and Block chain (CCSB), Shenzhen, China, 2024.
https://ieeexplore.ieee.org/document/10735594

5) B. H. R. D. Reddy, A. K. Veedu and B. BM, "Block chain Technology in Agriculture," 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS), Bengaluru, India, 2024. https://ieeexplore.ieee.org/document/10816748.

6) Block chain technology for sustainable supply chains: A comprehensive review and future prospects March 2024 World Journal of Advanced Research and Reviews. https://ieeexplore.ieee.org/document/10250707

7) K. R. Chaganti, B. N. Kumar, P. K. Gutta, S. L. Reddy Elicherla, C. Nagesh and K. Raghavendar, "Blockchain Anchored Federated Learning and Tokenized Traceability for Sustainable Food Supply Chains," 2024.
https://ieeexplore.ieee.org/document/10866271

8) M. Alsamman, Y .Fazeaand F. Mohammed, "Towards a Block chain- Based Agricultural Ecosystem: A systematic review," 2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), Sakheer, Bahrain, 2023. https://ieeexplore.ieee.org/document/10391332

9) Subramanian, B. Selvaraj, R. Sivakumar, R. Tabassum and S. Rajaram, "Enhancing Supply Chain Traceability with Block chain Technology: A Study on Dairy, Agriculture, And Sea food Supply Chains, " 2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 2023. https://ieeexplore.ieee.org/document/10265785

10) P. V. Chate, S. B. Mane and R. Y. Sarode, "An Efficient Approach For Sustainable Agricultural Trading Using Block chain Technology," 2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS), Kochi, India,2022. https://ieeexplore.ieee.org/document/9885711

11) Singh Bist et al., "AI-Enabled Block chain for Supply Chain in Agriculture," 2022 IEEE Creative Communication and Innovative Technology (ICCIT), Tangerang, Indonesia, 2022. https://ieeexplore.ieee.org/document/10118743

12) Shivendra, K. Chiranjeevi, M. K. Tripathiand D. D. Maktedar, "Block chain Technology in Agriculture Product Supply Chain,"2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), Coimbatore, India, 2021. https://ieeexplore.ieee.org/document/9395886

13) G. K. Akella, S. Wibowo, S. Grandhi and S. Mubarak, "Design of a Block chain- based Decentralized Architecture for Sustainable Agriculture: Research-in-Progress,"2021 IEEE /ACIS 19th International Conference on Software Engineering Research, Management and Applications (SERA), Kanazawa, Japan, 2021. https://ieeexplore.ieee.org/document/9509044

14) Block chain in agriculture January 2021 DOI:10.1016/B978-0-12-821470-1.00003-3 https://www.researchgate.net/publication/348904679

15) Kang, P., & Indra-Payoong, N.(2021). A Frame work of Block chain Smart Contract for Sustainable Agri-Food Supply Chain. *INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND TECHNOLOGY (ISJET)*, *5*(2), 1–14. https://ph02.tci-thaijo.org/index.php/isjet/article/view/242212

16) Demestichas, KonstantinosP., and EvgeniaF. Adamopoulou. "Block chain in Agriculture Traceability Systems: A Review." Applied Sciences (2020)

17) C. YU, Y. ZHAN and Z. LI, "Using Block chain and Smart Contract for Traceability in Agricultural Products Supply Chain," 2020 International Conference on Internet of Things Intelligent Applications (ITIA), Zhenjiang, China, 2020. https://ieeexplore.ieee.org/document/9312315

18) W. Lin et al., "Block chain Technology in Current Agricultural Systems: From Techniques to Applications," in IEEE Access, vol. 8, pp. 143920-143937, 2020. https://ieeexplore.ieee.org/document/9159588

19) K. Salah, N. Nizamuddin, R. Jayaramanand M.Omar, "Block chain- Based Soybean Traceability in Agricultural Supply Chain," in IEEE Access, vol. 7, pp. 73295-73305, 2019. https://ieeexplore.ieee.org/document/8718621

20) J. Liand X. Wang, "Research on the Application of Block chain in the Traceability System of Agricultural Products," 2018. https://ieeexplore.ieee.org/document/8469456